

Abstract

The diagnosis of breast tumours using fine needle aspiration (FNA) offers a less invasive alternative to biopsy, but reliable classification remains essential. This study compares a multilayer perceptron (MLP) and a linear support vector machine (SVM) on the Breast Cancer Wisconsin (Diagnostic) dataset to evaluate whether increased model complexity improves performance. Both models were trained on standardised features using 5-fold stratified cross-validation for hyperparameter optimisation, with performance assessed using accuracy, precision, recall, and F1 score. Despite the MLP's ability to learn non-linear feature interactions, both models achieved identical test performance, reaching 97.4% accuracy with matching precision, recall, and F1 scores. These results suggest that the dataset can be effectively separated using a linear decision boundary, limiting the benefit of additional model complexity. The findings highlight that, in structured biomedical datasets, simpler linear models can perform comparably to more flexible neural networks while offering advantages in interpretability.

1. Introduction

Breast tumour diagnosis traditionally relies on invasive biopsy procedures. Fine needle aspiration offers a less invasive alternative but can be inconsistent and clinician dependent. To improve accuracy and objectivity, Street et al. (1993) developed a machine learning system using features from digitised FNA images, forming the Breast Cancer Wisconsin (Diagnostic) dataset [1]. The dataset contains 569 samples described by 30 features, with prior work reporting high accuracy (97–99%) for both SVM and MLP models [6,7]. This study compares two approaches to the classification task: a multilayer perceptron (MLP) and a linear support vector machine (SVM). The MLP learns non-linear decision boundaries, whereas the linear SVM applies a fixed linear boundary. This comparison highlights the trade-off between model flexibility and simplicity in learning from biomedical data.

The central hypothesis is that the MLP's ability to learn non-linear feature interactions may offer an advantage over the linear SVM. However, although the dataset is not strictly linearly separable, prior work shows that high accuracy can be achieved with a single linear decision boundary (2). This suggests that both models may perform similarly, making the comparison primarily about the value of added model complexity.

2. Methods

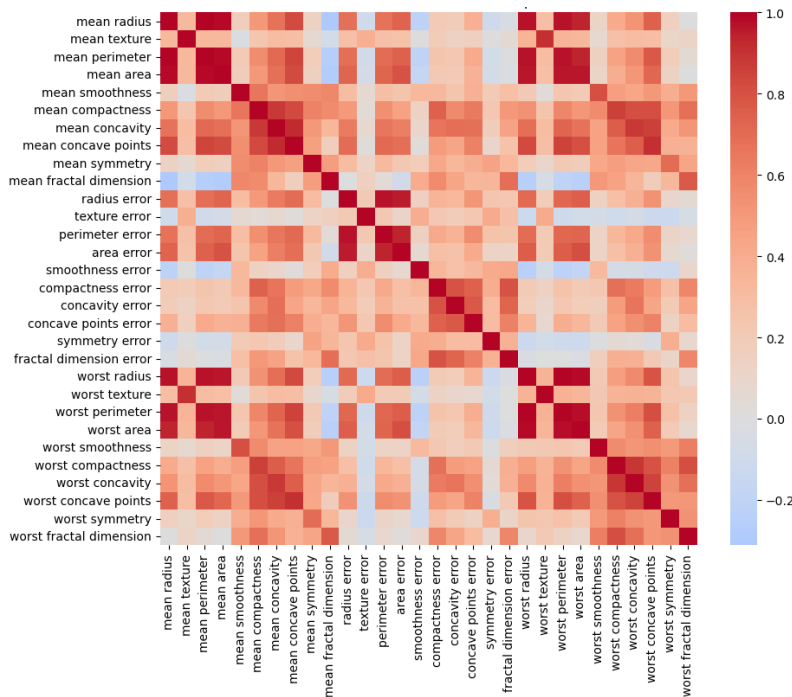
2.1 Dataset and Preprocessing

The dataset was loaded from scikit-learn's built-in datasets module. As shown in *Table 1*, features vary substantially in scale, so standardisation is needed to stop larger values from dominating the model. Strong correlations exist between related features (e.g. radius, perimeter, area) (*Figure 1*), which can affect learning efficiency and model stability. Standardisation was applied using training data only to prevent leakage. The data was split into 80% training and 20% test using stratified splitting to preserve the class ratio across splits. Labels were represented as {0, 1} for the MLP, compatible with Binary Cross-Entropy loss, and converted to {-1, +1} for the SVM, required for hinge loss.

Table 1: Summary Statistics

Feature	Mean	Std	Min	Max
mean radius	14.13	3.52	6.98	28.11
mean texture	19.29	4.3	9.71	39.28
mean perimeter	91.97	24.3	43.79	188.5
mean area	654.89	351.91	143.5	2501.0
mean smoothness	0.096	0.014	0.053	0.163
mean compactness	0.104	0.053	0.019	0.345
mean concavity	0.089	0.08	0.0	0.427
mean concave points	0.049	0.039	0.0	0.201
mean symmetry	0.181	0.027	0.106	0.304
mean fractal dim.	0.063	0.007	0.05	0.097
radius error	0.405	0.277	0.112	2.873
texture error	1.217	0.552	0.36	4.885
perimeter error	2.866	2.022	0.757	21.98
area error	40.34	45.49	6.8	542.2
smoothness error	0.007	0.003	0.002	0.031
compactness error	0.025	0.018	0.002	0.135
concavity error	0.032	0.03	0.0	0.396
concave pts error	0.012	0.006	0.0	0.053
symmetry error	0.021	0.008	0.008	0.079
fractal dim. error	0.004	0.003	0.001	0.03
worst radius	16.27	4.83	7.93	36.04
worst texture	25.68	6.15	12.02	49.54
worst perimeter	107.26	33.6	50.41	251.2
worst area	880.58	569.36	185.2	4254.0
worst smoothness	0.132	0.023	0.071	0.223
worst compactness	0.254	0.157	0.027	1.058
worst concavity	0.272	0.209	0.0	1.252
worst concave pts	0.115	0.066	0.0	0.291
worst symmetry	0.29	0.062	0.157	0.664
worst fractal dim.	0.084	0.018	0.055	0.208

Figure 1: Feature Correlation Heatmap



2.2 MLP Architecture and Training

The MLP was implemented as a fully-connected network in PyTorch consisting of an input layer of 30 neurons, two hidden layers with ReLU activation, and a single output neuron with Sigmoid activation for binary classification. ReLU activation was used to introduce non-

linearity and support stable gradients, as they are less prone to saturation [5]. A sigmoid output produces probabilities for binary classification. The hidden layer widths, learning rate, and weight decay were optimised using 5-fold stratified cross-validation, with the final configuration selected as described in **Section 3.1**.

Training used Binary Cross-Entropy loss with SGD and momentum (0.9) to smooth updates [8]. L2 regularisation was applied via weight decay in the optimiser, penalising large weights to reduce the risk of overfitting. Early stopping with a patience of 20 epochs monitored validation loss and restored the best weights when patience was exhausted. This training approach follows the backpropagation framework established by Rumelhart, Hinton, and Williams (1986) [3].

2.3 Linear SVM Architecture and Training

The linear SVM was implemented as a single linear layer in PyTorch with 30 input neurons and one output neuron, producing a raw score with no activation function. The model learns a linear decision boundary, with classification determined by the sign of the output. Predictions are determined by the sign of the output score: positive scores map to Benign (+1) and negative scores to Malignant (−1). The C parameter and learning rate were optimised using 5-fold stratified cross-validation, with the final configuration selected as described in Section 3.1.

L2 regularisation was implemented via weight decay ($1/C$), reflecting the SVM formula where C controls the trade-off between margin width and classification error [4]. As weight decay is derived from C, it was treated as the sole regularisation parameter. The model was optimised using SGD with momentum (0.9), which smooths updates using past gradients. Early stopping with a patience of 20 epochs monitored validation hinge loss and restored the best weights when patience was exhausted, applied identically to the MLP for consistency.

3. Experiments

3.1 Hyperparameter Optimisation

Both models used 5-fold stratified cross-validation, with standardisation applied inside each fold to prevent leakage from validation data into the scaler. Five folds were selected to balance bias and variance on a relatively small dataset. For the MLP, a grid search was conducted across three hidden layer configurations: (64,32), (128,64), and (32,16) neurons. Three learning rates [0.001, 0.01, 0.1] and three weight decay values [0.0001, 0.001, 0.01] were also searched, giving 27 configurations across 135 training runs. For the SVM, C values [0.1, 1.0, 10.0] and learning rates [0.001, 0.01, 0.1] were searched, giving 9 configurations across 45 training runs. In both cases, the configuration with the lowest mean validation loss across all folds was selected for final training.

3.2 Evaluation Metrics

Four metrics were used to evaluate model performance: accuracy, precision, recall, and F1 score. Accuracy, precision, recall, and F1 score were used. Recall is particularly important, as failing to detect malignant cases carries greater risk. Malignant class recall was computed separately by targeting class 0 directly, using scikit-learn's recall score function with `pos_label=0`. All metrics were computed on the held-out test set, which was not used at any point during training or model selection.

4. Results

Figure 2 shows the effect of weight decay and momentum on MLP training. Higher weight decay leads to better generalisation, and momentum of 0.9 substantially reduces the epochs needed to reach convergence compared to no momentum. *Figure 3* shows the equivalent results for the Linear Lower C values led to unstable training, while higher values produced smooth convergence. Momentum (0.9) improved convergence speed. *Figure 4* shows the cross-validation loss heatmap for the best MLP architecture, plotting learning rate against

weight decay. Learning rates below 0.01 failed to converge within the training budget across all weight decay values. The combination of $lr=0.1$ and $wd=0.001$ produced the best result, with a mean validation BCE loss of 0.0652. *Figure 5* shows the corresponding results for the Linear SVM, plotting C against learning rate. Larger C values consistently reduced validation hinge loss. The combination of $C=10.0$ and $lr=0.1$ produced the best result, with a mean validation hinge loss of 0.0704.

Figure 4: MLP Cross-Validation Loss Heatmap for Best Architecture

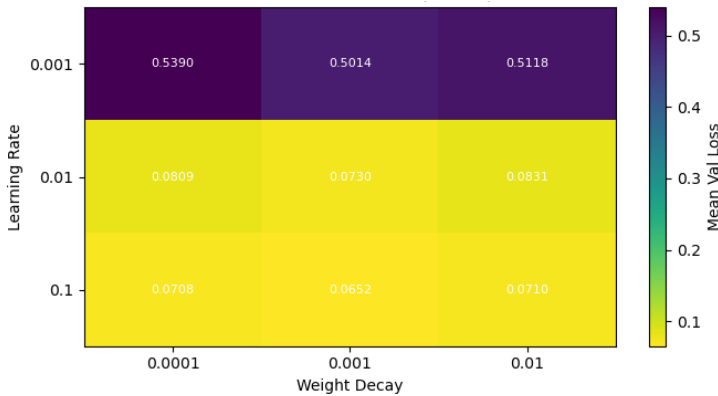
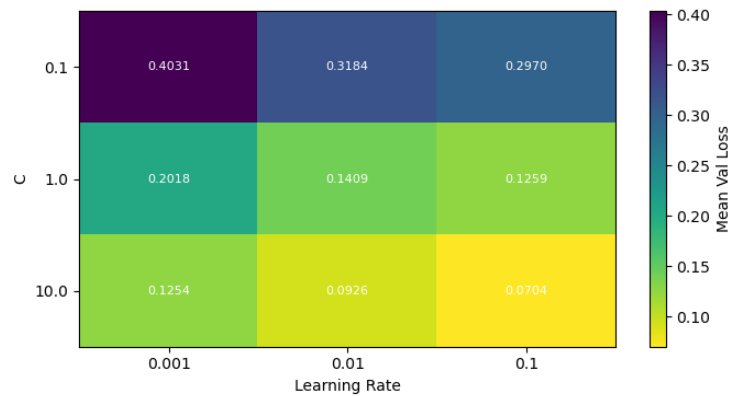


Figure 5: Linear SVM Cross-Validation Loss Heatmap for Best Architecture



Both models were retrained with their best hyperparameters on the full training set, using a 10% internal holdout for early stopping only. *Figure 6* shows the MLP training and validation loss curves. The best weights were saved at epoch 23 and early stopping triggered at epoch 46. After epoch 23, training loss continued to fall while validation loss began to rise. This is a sign of overfitting beginning to develop. The weight decay regularisation kept this gap small, meaning the model still performed well on unseen data. *Figure 7* shows the SVM training and validation loss curves. The best weights were saved at epoch 18, with early stopping at epoch 38. Unlike the MLP, validation loss remained consistently lower than training loss throughout. This is because the SVM has only 31 parameters compared to several hundred in the MLP, making it much less likely to overfit the training data. The SVM converged quickly and stably throughout.

Figure 6: MLP Final Model Training and Validation Loss Curves
(Hidden=(32,16), $LR=0.1$, $WD=0.001$)

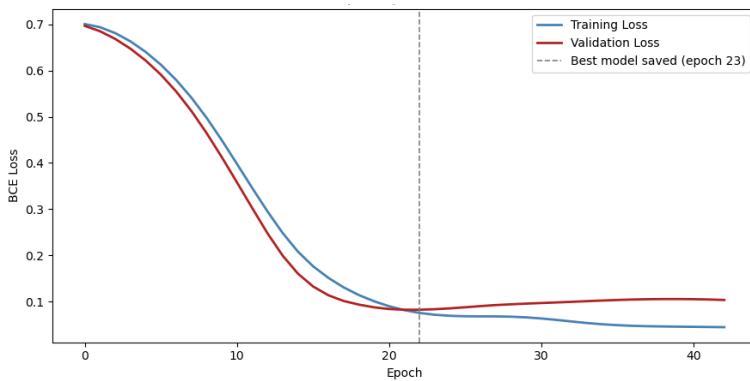


Figure 7: Linear SVM Final Model Training and Validation Loss Curves
($C=10.0$, $LR=0.1$)

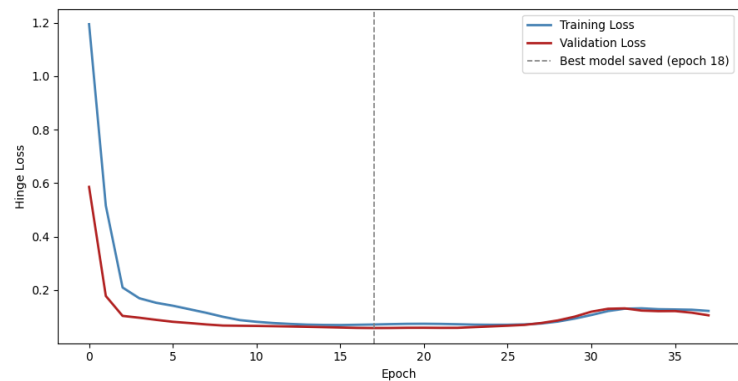


Table 2 summarises test set performance. Both models achieved 97.4% accuracy. Published results on this dataset report 99.12% [6] and 99.03% [7], using solvers rather than

gradient descent, and much larger MLP architecture of 500 nodes per layer. The small difference is expected given these methodological choices and the results remain well within the published range. The confusion matrices in Figures 8 and 9 show that both models produced the same overall error pattern. Both correctly identified 40 of 42 malignant cases and misclassified 1 benign case as malignant.

Table 2: Test Set Performance

Metric	MLP	Linear SVM
Accuracy	0.9737	0.9737
Precision	0.9726	0.9726
Recall	0.9861	0.9861
F1 Score	0.9793	0.9793
Malignant Recall	0.9524	0.9524
Training Time	0.02s	0.06s

Figure 8: MLP Confusion Matrix

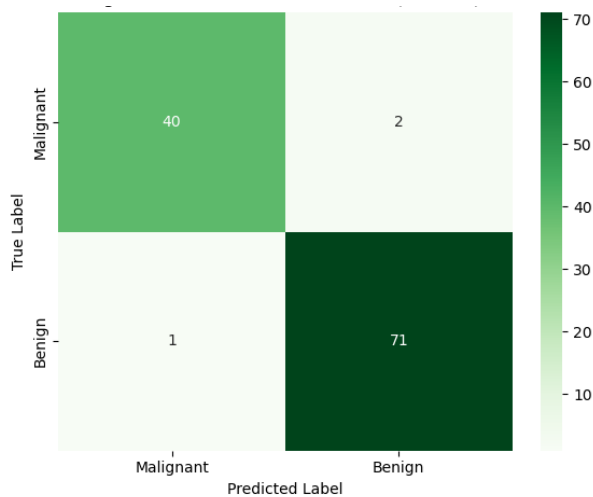
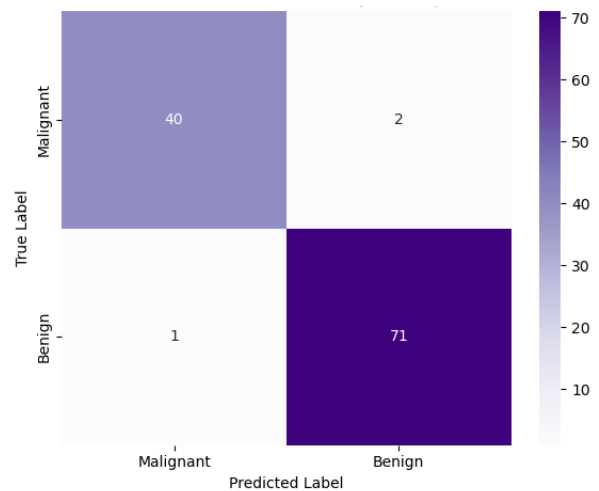


Figure 9: Linear SVM Confusion Matrix



5. Discussion

Both models achieved identical performance across all metrics, consistent with prior work showing that a linear boundary is sufficient for this dataset [2]. This suggests that the structure of the data limits the benefit of additional model complexity, as the classification task can already be effectively solved with a linear decision boundary.

Although the aggregate results are identical, the models differ in how their predictions are represented. The MLP produces probabilistic outputs, allowing the classification threshold to be adjusted post-training, which may be useful in applications where sensitivity can be prioritised. In contrast, the SVM produces a raw score, with classification determined by its sign, offering less flexibility in thresholding.

The models also differ in interpretability. The SVM assigns a single weight to each feature, making its decisions directly traceable to input variables. The MLP distributes its representation across multiple layers, making individual predictions less transparent. This distinction may be relevant in clinical settings where understanding feature contributions is important.

Differences in training behaviour were also observed. The MLP showed mild overfitting, while the SVM generalised consistently throughout training. This reflects the higher capacity of the MLP compared to the simpler SVM, which is less prone to fitting noise in the data.

Despite having fewer parameters, the SVM required slightly longer training due to noisier gradients from hinge loss.

A key limitation of this study is the dataset itself. With a relatively small sample size and well-structured feature space, both models reach a performance ceiling defined by the data rather than model capability. This is supported by the finding that smaller MLP architectures performed better than larger ones. On more complex datasets with non-linear structure, the advantage of the MLP would be expected to be more pronounced.

6. Conclusion

This study compared an MLP and linear SVM on the Breast Cancer Wisconsin dataset. Both models achieved identical performance, indicating that a linear boundary is sufficient for this task. The MLP's additional flexibility did not provide measurable benefit, while the SVM achieved comparable results with fewer parameters and greater interpretability. Future work should evaluate these models on more complex datasets where non-linear structure is more prominent.

References

- Agarap, A.F.M. (2018). On breast cancer detection. *Proceedings of the 2nd International Conference on Machine Learning and Soft Computing - ICMLSC '18*. [online] doi:<https://doi.org/10.1145/3184066.3184080>. [7]
- Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), pp.273–297. doi:<https://doi.org/10.1007/BF00994018>. [4]
- Glorot, X. and Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. [online] proceedings.mlr.press. Available at: <https://proceedings.mlr.press/v9/glorot10a.html>. [5]
- Jelassi, S. and Li, Y. (2022). *Towards understanding how momentum improves generalization in deep learning*. [online] arXiv.org. Available at: <https://arxiv.org/abs/2207.05931> [Accessed 9 Mar. 2026]. [8]
- Mangasarian, O.L., Street, W.N. and Wolberg, W.H. (1995). Breast Cancer Diagnosis and Prognosis Via Linear Programming. *Operations Research*, [online] 43(4), pp.570–577. doi:<https://doi.org/10.1287/opre.43.4.570>. [2]
- Md. Murad Hossin, Javed, M., Md Rifat Bhuiyan, Rabea Akter Hira, Khan, T. and Molla, S. (2023). Breast cancer detection: an effective comparison of different machine learning algorithms on the Wisconsin dataset. 12(4), pp.2446–2456. doi:<https://doi.org/10.11591/eei.v12i4.4448>. [6]
- Rumelhart, D.E., Hinton, G.E. and Williams, R.J. (1986). Learning representations by back-propagating errors. *Nature*, [online] 323(6088), pp.533–536. doi:<https://doi.org/10.1038/323533a0>. [3]
- Street, W.N., Wolberg, W.H. and Mangasarian, O.L. (1993). Nuclear feature extraction for breast tumor diagnosis. *Biomedical Image Processing and Biomedical Visualization*. doi:<https://doi.org/10.1117/12.148698>. [1]